

File Types

1 Basic Idea

Definition

File type describes the role or interpretation of a file, such as regular file, directory, special device file, ASCII text file, or binary file.

- Many operating systems support more than one kind of file.
- UNIX-like systems and Windows support regular files and directories.
- UNIX also supports character special files and block special files.
- This note mainly focuses on regular files.
- Regular files contain user information.

2 Major File Categories

File type	Meaning
Regular file	Contains ordinary user information, such as text, programs, images, or data.
Directory	System file used to maintain the structure of the file system.
Character special file	Models serial I/O devices such as terminals, printers, and networks.
Block special file	Models block devices such as disks.

Core Point

Regular files store user data, while directories and special files help the operating system organize names and model devices.

3 Regular Files

Regular File

A **regular file** is a file that contains user information rather than directory metadata or device access information.

- Regular files may contain text.
- They may contain executable programs.
- They may contain images, audio, video, archives, or application data.
- The operating system may treat the file as a byte sequence.

- Programs usually interpret the actual contents.
- Regular files are commonly divided into ASCII files and binary files.

4 ASCII Files

ASCII File

An **ASCII file** is a regular file made of lines of text.

- ASCII files contain printable text characters.
- Lines do not have to be the same length.
- Some systems terminate lines with a carriage return.
- Some systems terminate lines with a line feed.
- Windows traditionally uses both carriage return and line feed.
- ASCII files can be displayed, printed, and edited easily.

Main Advantage

ASCII files are portable and easy for both humans and programs to process.

5 Why ASCII Files Are Useful

- They can be edited with any text editor.
- They can be printed directly.
- They are easy to inspect for debugging.
- They work well as input and output for many programs.
- Shell pipelines are easier to use when programs exchange text.
- Standard text conventions make program output easier for another program to interpret.

6 Binary Files

Binary File

A **binary file** is a regular file that is not an ASCII text file and usually has an internal structure understood by specific programs.

- Binary files may contain arbitrary byte values.
- Printing a binary file as text usually produces unreadable output.

- Binary files often contain numeric fields, headers, offsets, tables, or encoded data.
- The operating system may recognize some binary formats.
- Programs that use the file know how to interpret its internal structure.

Do Not Print Blindly

Sending a binary file directly to a printer or terminal may produce meaningless characters.

7 Binary File Examples

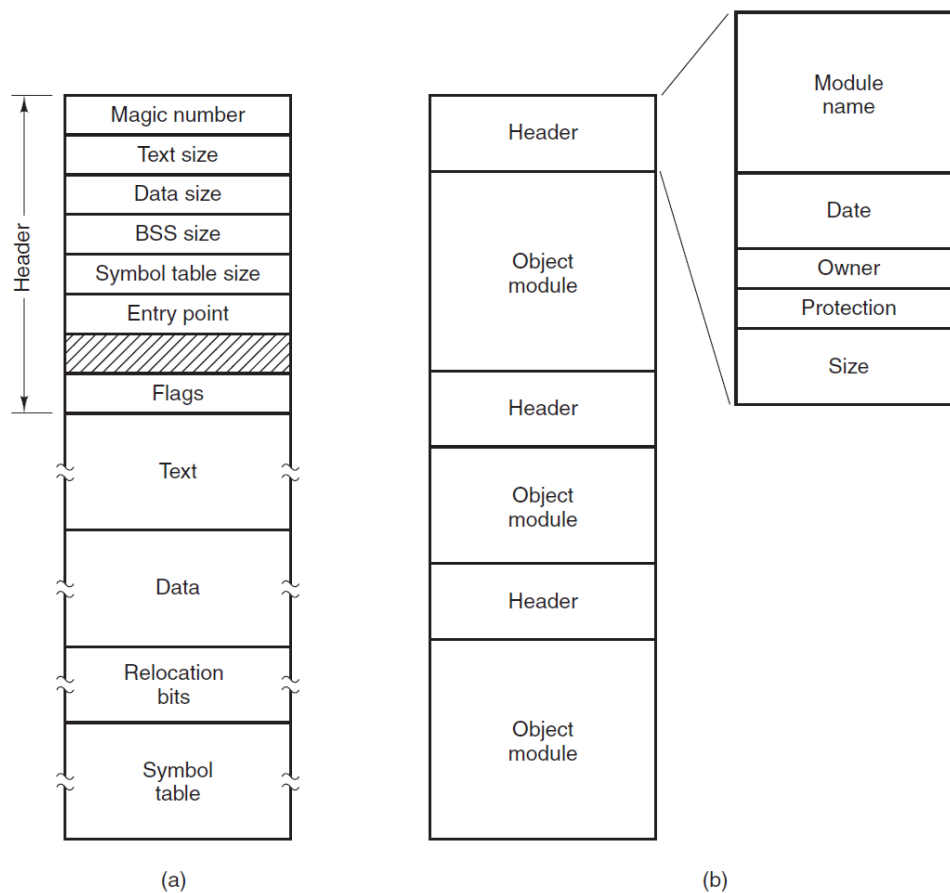


Figure 4-3. (a) An executable file. (b) An archive.

Binary file	Internal structure
Executable file	Contains a header, program text, data, relocation bits, and symbol table.
Archive file	Contains multiple object modules, each preceded by a header.

8 Executable Binary Files

Executable File

An **executable file** is a binary file whose format is recognized by the operating system as runnable program code.

- The OS normally executes a file only if it has the expected executable format.
- The header starts with a **magic number**.
- The magic number identifies the file as an executable and helps prevent accidental execution of non-executable data.
- The header includes sizes of file sections.
- It also includes the program entry point and flags.
- Program text and data are loaded into memory.
- Relocation bits help adjust addresses when loading.
- The symbol table is useful for debugging.

9 Executable File Sections

Section	Purpose
Magic number	Identifies the file as an executable.
Text size	Gives the size of the program instruction section.
Data size	Gives the size of initialized data.
BSS size	Gives the size of uninitialized data.
Symbol table size	Gives the size of debugging/linking symbols.
Entry point	Tells where execution begins.
Flags	Stores executable options or status bits.
Text	Contains program instructions.
Data	Contains program data.
Relocation bits	Used to adjust addresses during loading.
Symbol table	Used mainly for debugging.

10 Archive Binary Files

Archive

An **archive file** is a binary file containing a collection of compiled modules that have not necessarily been linked into one executable.

- UNIX archives can store library procedures or object modules.
- Each module is preceded by a header.
- The header contains metadata about that module.

- This metadata is stored as binary fields.
- Printing the archive as plain text produces unreadable output.

Archive header field	Meaning
Module name	Name of the object module.
Date	Creation or modification date.
Owner	Owner of the module.
Protection	Access protection information.
Size	Size of the module.

11 OS Recognition of File Types

- Every operating system must recognize at least its own executable file type.
- Some operating systems recognize additional file types.
- Recognizing executable format is necessary for safe program loading.
- Some older systems strongly enforced file-type rules.
- TOPS-20 even checked whether source files had changed since the binary was built.
- If the source was newer, it could automatically recompile before running.

12 Strongly Typed Files

Strong Typing

A system has **strongly typed files** when the OS enforces file-type rules, often using mandatory extensions or metadata.

- Strong typing can help beginners avoid mistakes.
- It can also prevent unexpected or creative uses of files.
- Example: a formatter reads a `.c` source file and writes formatted output as `.dat`.
- If the OS refuses to compile `.dat`, the user cannot easily use the formatted output as C source.
- If the OS also refuses to rename `file.dat` to `file.c`, the user is blocked by the type system.

Trade-Off

Strong file typing may protect novice users, but it can frustrate experienced users who need flexibility.

13 ASCII vs. Binary

Feature	ASCII file	Binary file
Contents	Lines of human-readable text.	Arbitrary bytes with program-defined structure.
Display	Can usually be printed or displayed directly.	Usually unreadable if printed directly.
Editing	Can be edited by text editors.	Needs specialized tools or programs.
Interchange	Good for pipelines and manual inspection.	Efficient and structured for specific software.

Final Point

File types help systems and programs decide how to interpret file contents, but too much OS enforcement can reduce flexibility.